Getting Started Guide

Sysnex Labs

January 2026

Getting Started Guide

Complete Onboarding for NexSuite: Installation, Configuration, First Model, and CI/CD Integration

Version: 1.0 **Date**: January 2026 **Target Audience**: New NexSuite Users **Est. Time**: 1-2 hours

---

Table of Contents

Prerequisites

System Requirements

Component	Minimum	Recommended
OS	Windows 10, macOS 10.15, Ubuntu 20.04	Windows 11, macOS 13+, Ubuntu 22.04
RAM	4 GB	8 GB+
Disk Space	500 MB	2 GB
CPU	Dual-core	Quad-core+

Required Software

1. **Visual Studio Code** (v1.80+)
   - Download: https://code.visualstudio.com/
2. **Git** (v2.30+)
   - Windows: https://git-scm.com/download/win
   - macOS: `brew install git`
   - Linux: `sudo apt install git`
3. **Optional: GitHub CLI** (for PR workflows)
   - Download: https://cli.github.com/
   - Install: `brew install gh` (macOS) or `winget install gh` (Windows)

Installation

Step 1: Install NexSuite VS Code Extension

Option A: VS Code Marketplace (Recommended)

1. Open VS Code
2. Press `Ctrl+Shift+X` (Windows/Linux) or `Cmd+Shift+X` (macOS)
3. Search for "NexSuite SysML v2 LSP"
4. Click **Install**

Option B: Command Line

```
code --install-extension nexsuite.sysml-v2-lsp
```

Option C: VSIX Package (Offline Installation)

1. Download VSIX from https://github.com/SysnexLabs/nexsuite/releases
2. In VS Code: `Extensions` → `...` → `Install from VSIX...`
3. Select downloaded `.vsix` file

Step 2: Verify Installation

1. Open VS Code
2. Create new file: `test.sysml`
3. Type:

```
part def Vehicle {
    attribute mass : Real;
```

```
}
```

4. Verify:
   - ✅ Syntax highlighting (colored keywords)
   - ✅ No red squiggles (diagnostics working)
   - ✅ Status bar shows "SysML v2 LSP" (bottom right)

Success: NexSuite is installed and running!

Step 3: Configure Extension (Optional)

Open Settings: `File` → `Preferences` → `Settings` → Search "SysML"

Recommended Settings:

Setting	Default	Description
`sysml.lsp.logLevel`	`info`	Set to `debug` for troubleshooting
`sysml.lsp.standardLibrary`	`auto`	Auto-downloads 402 standard library files
`sysml.diagnostics.enabled`	`true`	Enable real-time error checking
`sysml.formatting.indentSize`	`4`	Spaces per indent level

Optional: Enable ASPICE Validation

```
{
  "sysml.aspice.enabled": true,
  "sysml.aspice.rulesets": ["automotive", "iso15288"]
}
```

First Steps: Creating Your First Model

Step 1: Create Project Structure

```
# Create project directory
mkdir my-sysml-project
cd my-sysml-project

# Create folders
mkdir -p models/requirements
mkdir -p models/architecture
mkdir -p models/behavior

# Initialize Git
git init
git add .
git commit -m "Initial project structure"
```

Step 2: Create Your First Requirement

File: `models/requirements/vehicle-requirements.sysml`

```
/**
 * Vehicle Requirements Specification
 * Author: Your Name
 * Date: 2026-01-03
 */

package VehicleRequirements {

    requirement def VehicleSpeedRequirement {
        doc /*
         * REQ-001: The vehicle shall achieve 0-60 mph acceleration
         * in less than 6 seconds.
         */

        attribute id = "REQ-001";
        attribute priority = "High";

        subject vehicle : Vehicle;

        require constraint {
            vehicle.acceleration_0_to_60mph < 6.0 [s]
        }
    }

    requirement def VehicleRangeRequirement {
        doc /*
         * REQ-002: The vehicle shall achieve a minimum range of
         * 300 miles on a single charge.
         */

        attribute id = "REQ-002";
        attribute priority = "High";

        subject vehicle : Vehicle;

        require constraint {
            vehicle.electric_range > 300 [mi]
        }
    }
}
```

Save File: Press `Ctrl+S`

Observe: - ✅ Syntax highlighting - ✅ Real-time diagnostics (no errors) - ✅ Document outline (left sidebar: Explorer → Outline)

Step 3: Create Architecture Model

File: `models/architecture/vehicle-architecture.sysml`

```
package VehicleArchitecture {
    import VehicleRequirements::*;

    /**
     * Top-level vehicle part definition
     */
    part def Vehicle {
        // Attributes
        attribute mass : Real;
        attribute acceleration_0_to_60mph : Real;
        attribute electric_range : Real;
```

```
        // Parts
        part battery : Battery;
        part motor : ElectricMotor;
        part chassis : Chassis;
        part electronics : Electronics;

        // Connections
        connect motor.powerInput to battery.powerOutput;

        // Satisfactions
        satisfy VehicleSpeedRequirement;
        satisfy VehicleRangeRequirement;
    }

    part def Battery {
        attribute capacity : Real;  // kWh
        attribute voltage : Real;   // V

        port powerOutput : PowerInterface;

        constraint thermalLimit {
            temperature < 60 [degC]
        }

        attribute temperature : Real;  // degC
    }

    part def ElectricMotor {
        attribute power : Real;  // kW
        attribute torque : Real; // Nm

        port powerInput : PowerInterface;
    }

    part def Chassis {
        attribute weight : Real;
    }

    part def Electronics {
        attribute voltage : Real;
    }

    interface def PowerInterface {
        attribute voltage : Real;
        attribute current : Real;
    }
}
```

Save and Observe: - ✅ Cross-file references work (import statement) - ✅ Traceability (Vehicle satisfies requirements)

Step 4: Add Behavior Model

File: `models/behavior/vehicle-behavior.sysml`

```
package VehicleBehavior {
    import VehicleArchitecture::*;

    /**
     * Vehicle startup sequence
     */
```

```
    action def StartVehicle {
        in vehicle : Vehicle;

        action performSelfTest : SelfTest;
        then action initializeBattery : InitializeBattery;
        then action engageMotor : EngageMotor;
        then action displayReady : DisplayReadyMessage;
    }

    action def SelfTest {
        doc /* Perform vehicle self-test diagnostics */
    }

    action def InitializeBattery {
        doc /* Initialize battery management system */
    }

    action def EngageMotor {
        doc /* Engage electric motor */
    }

    action def DisplayReadyMessage {
        doc /* Display "Vehicle Ready" message to driver */
    }
}
```

Save and Observe: - ✅ Action flow (then succession) - ✅ Behavior modeling

---

Using LSP Features

Code Completion

Trigger: Type `part` and press `Ctrl+Space`

Result: Completion suggestions appear: - `part` (keyword) - `part def` (definition) - `part usage` (usage)

Example: 1. Type: `part` + `Ctrl+Space` 2. Select: `part def` 3. Type: `Transmission` + `Enter` 4. Result: `sysml part def Transmission {  |  // Cursor here  }`

Go-to-Definition

Example: 1. In `vehicle-architecture.sysml`, place cursor on `Battery` (line 24) 2. Press `F12` (or `Ctrl+Click`) 3. Result: Jump to `Battery` definition (line 31)

Cross-File Navigation: 1. Place cursor on `VehicleSpeedRequirement` (line 29) 2. Press `F12` 3. Result: Jump to `vehicle-requirements.sysml` (line 8)

Find References

Example: 1. Place cursor on `Vehicle` definition (line 12) 2. Press `Shift+F12` 3. Result: References panel shows: - `vehicle-requirements.sysml:15` (subject vehicle : Vehicle) - `vehicle-requirements.sysml:25` (subject vehicle : Vehicle) - `vehicle-behavior.sysml:10` (in vehicle : Vehicle)

Hover Information

Example: 1. Hover over `Battery` (line 24) 2. Result: Tooltip shows: `part def Battery  Attributes:  - capacity : Real  // kWh  - voltage : Real  // V  - temperature : Real  // degC`

Diagnostics (Error Checking)

Example (Introduce Error):

```
part def Vehicle {
    part engine : Engine;  // ❌ ERROR: Undefined type 'Engine'
}
```

Result: - Red squiggle under `Engine` - Error message: `Cannot resolve type 'Engine'` - Suggestion: `Did you mean 'ElectricMotor'?`

Fix:

```
part def Engine {
    // Define Engine first
}

part def Vehicle {
    part engine : Engine;  // ✅ OK
}
```

Refactoring (Rename Symbol)

Example: 1. Place cursor on `Battery` definition (line 31) 2. Press `F2` (Rename Symbol) 3. Type: `BatteryPack` 4. Press `Enter` 5. Result: All references to `Battery` renamed to `BatteryPack` across all files

---

Git Integration

Initialize Git Repository

```
cd my-sysml-project

# Initialize Git
git init

# Create .gitignore
cat > .gitignore <<EOF
# NexSuite
.sysml-cache/
.sysml-build/

# VS Code
.vscode/settings.json

# OS
.DS_Store
Thumbs.db
EOF
```

```
# Initial commit
git add .
git commit -m "Initial commit: Vehicle model"
```

Create GitHub Repository

Option A: GitHub CLI

```
gh repo create my-org/vehicle-model --private --source=. --remote=origin
git push -u origin main
```

Option B: GitHub Web UI

1. Go to https://github.com/new
2. Repository name: `vehicle-model`
3. Visibility: Private
4. Click "Create repository"
5. Follow instructions to push existing repository:

   ```
   git remote add origin https://github.com/my-org/vehicle-model.git
   git push -u origin main
   ```

Feature Branch Workflow

```
# Create feature branch
git checkout -b feature/add-transmission

# Edit model
code models/architecture/vehicle-architecture.sysml

# Add transmission
# ... (edit file) ...

# Commit changes
git add models/architecture/vehicle-architecture.sysml
git commit -m "Add transmission model with gear ratios"

# Push to remote
git push origin feature/add-transmission

# Create pull request
gh pr create --title "Add Transmission Model" \
             --body "Adds transmission with 6-speed gearbox"
```

Documentation Generation

Install Documentation Generator

NexSuite CLI:

```
cargo install sysml-cli
```

Or use pre-built binary: - Download from: https://github.com/SysnexLabs/nexsuite/releases - Extract to PATH

Generate MkDocs Documentation

```
# Generate documentation
sysml-cli docs --input models/ --output docs/ --format mkdocs

# Serve locally
cd docs
python -m mkdocs serve
```

Open Browser: http://localhost:8000

Generated Documentation: - Requirements specification (REQ-001, REQ-002) - Architecture diagrams (PlantUML) - Traceability matrix (Requirements → Design) - Model index (searchable)

Generate Sphinx Documentation (API-Style)

```
sysml-cli docs --input models/ --output docs-api/ --format sphinx

cd docs-api
make html

# Open in browser
open _build/html/index.html  # macOS
xdg-open _build/html/index.html  # Linux
start _build/html/index.html  # Windows
```

CI/CD Setup

GitHub Actions Workflow

Create Workflow File: `.github/workflows/model-validation.yml`

```
name: Model Validation

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  validate:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v3
```

```yaml
      - name: Install NexSuite CLI
        run: |
          cargo install sysml-cli

      - name: Syntax Validation
        run: |
          sysml-cli check models/**/*.sysml

      - name: Type Checking
        run: |
          sysml-cli check --types models/**/*.sysml

      - name: ASPICE Validation
        run: |
          sysml-cli verify --aspice models/**/*.sysml

      - name: Traceability Check
        run: |
          sysml-cli trace --gaps models/**/*.sysml

      - name: Generate Documentation
        run: |
          sysml-cli docs --input models/ --output docs/ --format mkdocs

      - name: Deploy Documentation (main branch only)
        if: github.ref == 'refs/heads/main'
        run: |
          cd docs
          python -m mkdocs gh-deploy --force

      - name: Upload Validation Report
        if: always()
        uses: actions/upload-artifact@v3
        with:
          name: validation-report
          path: validation-report.md
```

Commit Workflow:

```
git add .github/workflows/model-validation.yml
git commit -m "Add CI validation workflow"
git push
```

Observe: - GitHub Actions tab shows workflow running - All checks must pass before PR can merge

---

Validation on Every Commit

Pre-Commit Hook (Optional Local Validation):

Install pre-commit:

```
pip install pre-commit
```

Create `.pre-commit-config.yaml`:

```yaml
repos:
  - repo: local
    hooks:
```

```yaml
      - id: sysml-syntax
        name: SysML Syntax Validation
        entry: sysml-cli check
        language: system
        files: \.sysml$

      - id: sysml-types
        name: SysML Type Checking
        entry: sysml-cli check --types
        language: system
        files: \.sysml$
```

Install hooks:

```
pre-commit install
```

Result: Every commit validates syntax and types automatically

---

Next Steps

Recommended Learning Path

Week 1: SysML v2 Fundamentals - Read SysML v2 Primer: https://www.omg.org/spec/SysML/2.0/Primer - Complete interactive tutorial: https://sysml2.org/tutorial - Practice: Build simple models (requirements, architecture, behavior)

Week 2: Advanced Features - UVL Variability: https://docs.nexsuite.dev/uvl - Trade Studies: https://docs.nexsuite.dev/trade-studies - Constraint Solving: https://docs.nexsuite.dev/constraints

Week 3: Compliance Automation - ASPICE Work Products: https://docs.nexsuite.dev/aspice - ISO 26262 ASIL Tracking: https://docs.nexsuite.dev/iso26262 - Traceability: https://docs.nexsuite.dev/traceability

Week 4: Team Collaboration - Git Workflows: https://docs.nexsuite.dev/git - PR Reviews: https://docs.nexsuite.dev/pr-reviews - CI/CD Integration: https://docs.nexsuite.dev/cicd

---

Sample Projects

Automotive Examples: - ADAS (Advanced Driver Assistance): https://github.com/SysnexLabs/examples-adas - Electric Powertrain: https://github.com/SysnexLabs/examples-ev-powertrain - Braking System (ISO 26262): https://github.com/SysnexLabs/examples-braking

Aerospace Examples: - UAV (Unmanned Aerial Vehicle): https://github.com/SysnexLabs/examples-uav - Satellite Subsystems: https://github.com/SysnexLabs/examples-satellite

---

Community Resources

Discord Community: https://discord.gg/nexsuite - #getting-started (ask beginner questions) - #sysml-v2-help (language questions) - #aspice-iso26262 (compliance questions)

Office Hours: Weekly Q&A sessions (Wednesdays, 2pm ET) - Register: https://sysnexlabs.github.io/office-hours

YouTube Channel: https://youtube.com/@SysnexLabs - Video tutorials (Getting Started, LSP Features, CI/CD) - Webinar recordings

---

Training Workshops

1-Day Workshop: NexSuite Fundamentals ($500/person) - SysML v2 basics - NexSuite LSP features - Git workflows - Hands-on exercises

2-Day Workshop: ASPICE/ISO 26262 Automation ($1,200/person) - Requirements engineering - Traceability automation - ASIL tracking - Work product generation

Custom Training: Contact training@sysnex-labs.com

Troubleshooting

LSP Not Working

Symptoms: No syntax highlighting, no diagnostics

Solutions: 1. Check extension is installed: `Extensions` → Search "NexSuite" 2. Restart VS Code: `Ctrl+Shift+P` → "Reload Window" 3. Check LSP server status: `Ctrl+Shift+P` → "SysML: Show LSP Status" 4. View logs: `Ctrl+Shift+P` → "SysML: Show LSP Logs"

Standard Library Not Found

Symptoms: `Cannot resolve 'ScalarValues::Real'`

Solutions: 1. Download standard library: `Ctrl+Shift+P` → "SysML: Download Standard Library" 2. Verify path: Settings → `sysml.lsp.standardLibrary.path` 3. Manual download: https://github.com/Systems-Modeling/SysML-v2-Release/tree/master/sysml.library

---

Performance Issues

Symptoms: Slow LSP response (>500ms)

Solutions: 1. Exclude large directories: `.sysmlignore` file `# Exclude build/ .git/ node_modules/` 2. Increase memory limit: Settings → `sysml.lsp.maxMemoryMB` (default: 2048) 3. Disable unused features: Settings → `sysml.diagnostics.enabled` = false (temporary)

Support

Community Support (FREE)

- **GitHub Issues**: https://github.com/SysnexLabs/nexsuite/issues
- **Discord**: https://discord.gg/nexsuite
- **Documentation**: https://docs.nexsuite.dev

Enterprise Support

- **Email**: support@sysnex-labs.com
- **SLA**: 4-hour response (business hours)
- **Phone**: Available with Enterprise tier

Conclusion

Congratulations! You've completed the NexSuite Getting Started Guide.

What You've Learned: - ✅ Install and configure NexSuite - ✅ Create your first SysML v2 model (requirements, architecture, behavior) - ✅ Use LSP features (completion, navigation, diagnostics) - ✅ Set up Git workflows (branch, commit, PR) - ✅ Generate documentation (MkDocs, Sphinx) - ✅ Configure CI/CD validation (GitHub Actions)

Next Steps: 1. Explore sample projects: https://github.com/SysnexLabs/examples 2. Join Discord community: https://discord.gg/nexsuite 3. Attend weekly office hours (Wednesdays, 2pm ET) 4. Consider training workshop for deep dive

Welcome to the NexSuite community!

Contact Information

- **Website**: https://sysnexlabs.github.io
- **Email**: hello@sysnex-labs.com
- **GitHub**: https://github.com/SysnexLabs

NexSuite - Modern MBSE for Modern Teams